Binary arithmetic: Addition

#9

CS 0447
Introduction to
Computer Programming

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Luís Oliveira

Fall 2020

Class announcements

- nope

Binary addition

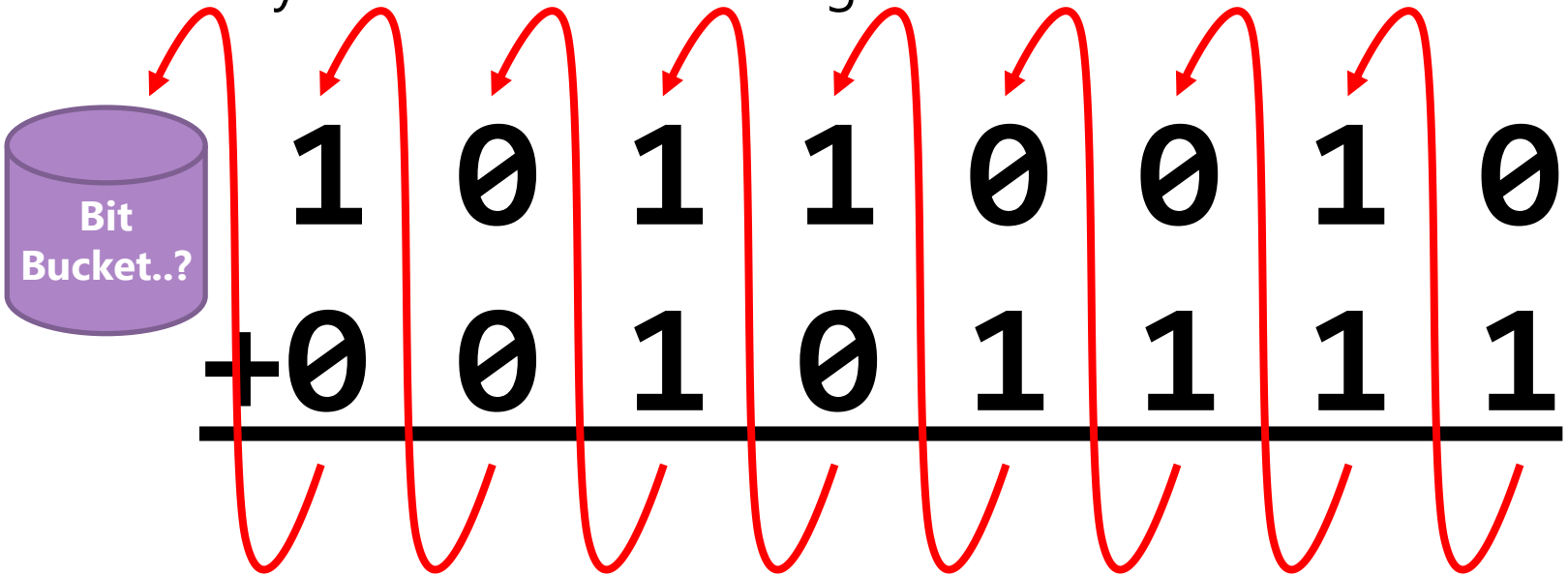
Adding in binary

- It works the same way as you learned in school
  - Except instead of carrying at 10_{10}, you carry at... 10_2!
    - 1 + 1 = $\mathbf{10_2}$ (2_{10})
    - 1 + 1 + 1 = $\mathbf{11_2}$ (3_{10})
- Let's try it. (what are these in decimal?)

```
  0 1 1 1   1 1 0
  1011 0010
+ 0010 1111
-----------
  1110 0001
```

Formalizing the algorithm

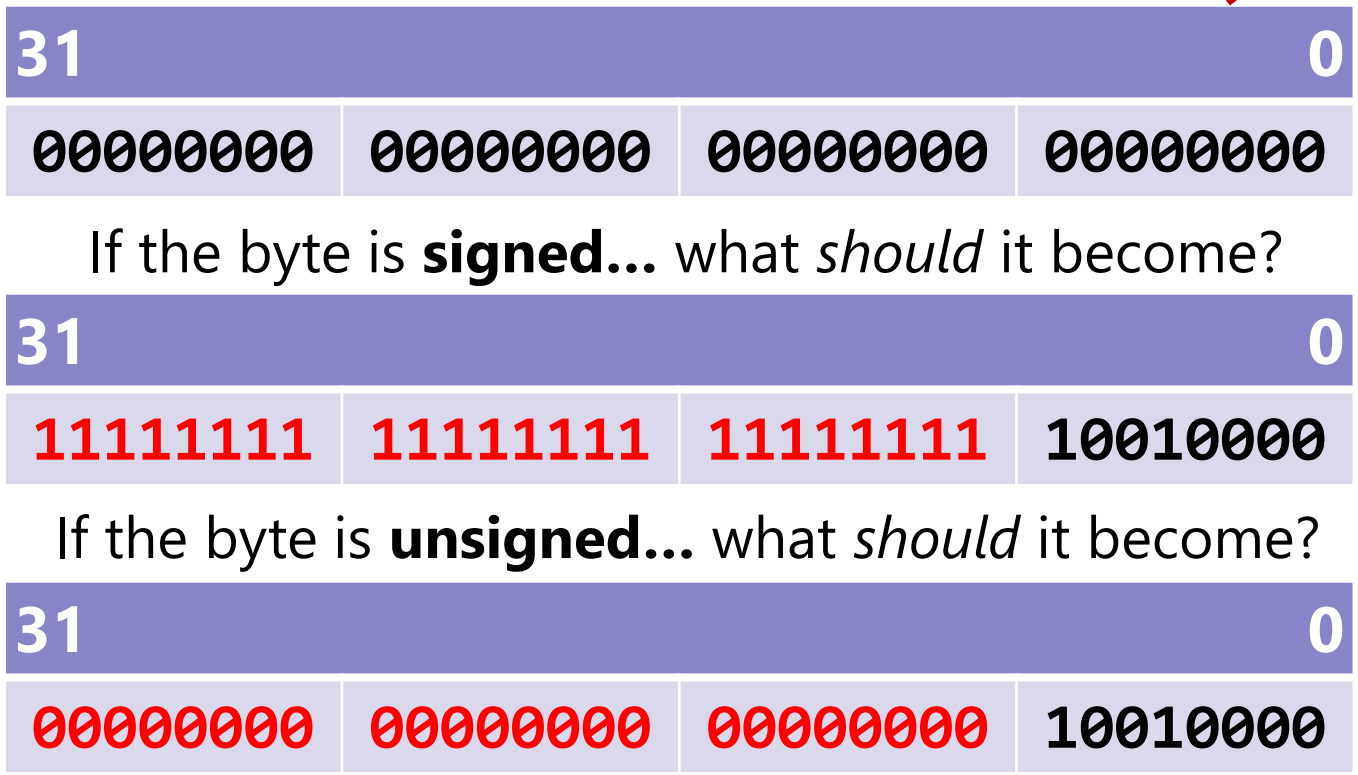
- for each pair of bits **starting at the LSB,**
  - add the two bits and the carry
  - the **low bit** of the sum goes into the sum row
  - the **high bit** of the sum is the carry for the **next higher bit**
- this is the **grade school algorithm**
  - cause it's how you learned to add in grade school



Signed Numbers

Remember this?

- if you load a **byte…**



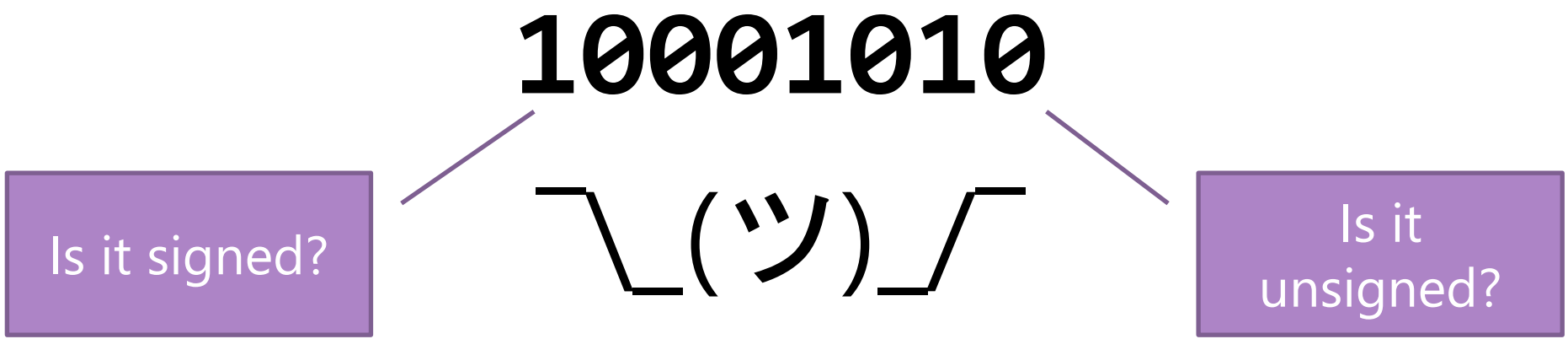


`lb` does sign extension.

`lbu` does zero extension.

Arbitrariness… Again

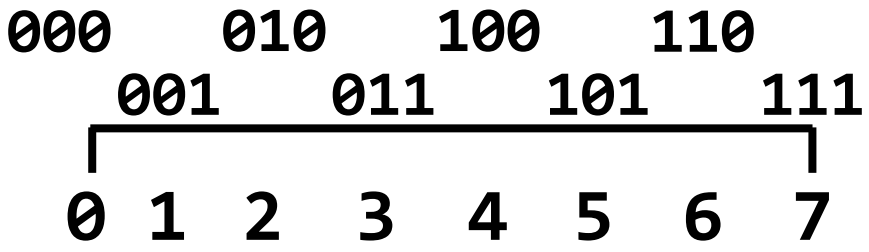
- What does this mean?



What's the difference?

Unsigned integers – they are all positive!

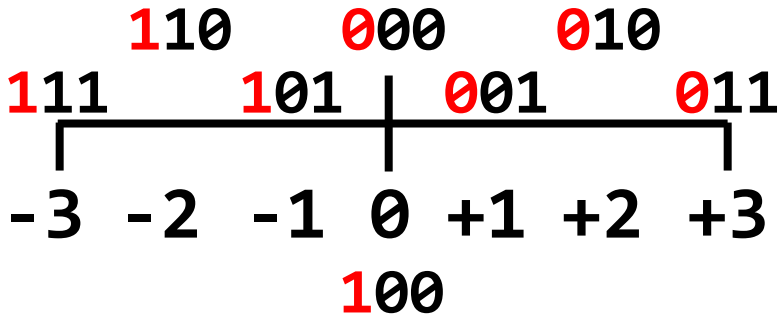
- We have numbers, they are all positive!
  - On a number line:



- What if we want to have negative numbers?

If you wanted to implement signed integers…

- What would be the most intuitive way of doing it?
  - You could have a "sign bit" where 0 means +, and 1 means -
- This is **sign-magnitude representation**. on a number line:



- What about the pattern **100**?
  - negative sign, 0 magnitude
  - **NEGATIVE ZERO??**
- Arithmetic is a bit awkward

Sign-magnitude arithmetic

- Negation

-(3) → 0011 → 1011

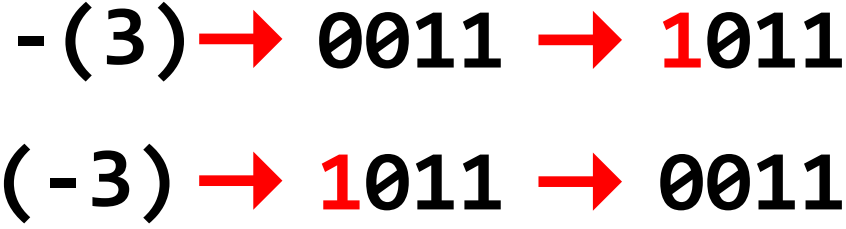
bit pattern for positive 3? *Flip MSB!*

-(-3) → 1011 → 0011

bit pattern for negative 3? *Flip MSB!*

Sign-magnitude arithmetic

- Negation



- (wrong) Addition

to binary?

```
  11
  3 → 0011
+ 3 →+0011
-----------
  6   0110
```

bit pattern for -3… positive 3?

```
  3
+-3
---
  0
```

→ 1011

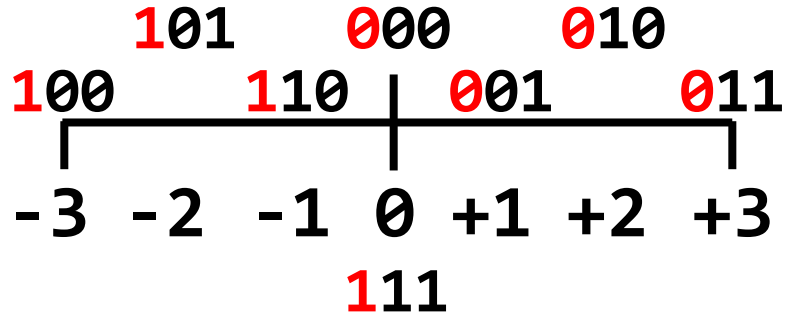
```
  11
 0011
+1011
------
 1110 → -6
```

to decimal?

- It's not impossible
  - But it is awkward

Well, what if…

- We use a technique borrowed from old accounting practices and mechanical calculators, we can **flip all the bits** to negate a number
- This is **ones' complement.** on a number line:
- We have the same problem with **111** as sign-magnitude had with **100**



- This *does* make arithmetic easier
- But it is not used in modern computers.

Sign-magnitude arithmetic

- Negation

-(3) → 0011 → 1100

bit pattern for positive 3? *flip!*

-(-3) → 1100 → 0011

bit pattern for negative 3? *flip!*

Sign-magnitude arithmetic

- Negation

-(3) → 0011 → 1100

-(-3) → 1100 → 0011

- Addition

to binary? 0010

3 → 0011

\+ 2 → +0010

5 0101

bit pattern for -2... positive 2?

3 0010

+-2 → *flip!* → 1101

1 0001

1111

0011

+1101

0000???

+0001

0001

1

Add the carry back in

this is positive, so don't flip

to decimal?

One’s complement arithmetic

- Do you really want to write code like

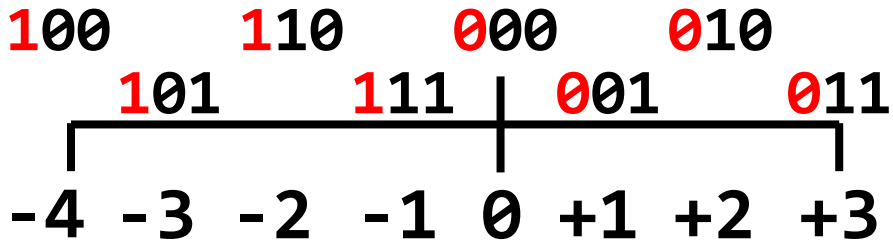
```
if( (my_var == 0) OR (my_var == -0) )
{
        …
}
```

- Just kidding… Compilers would handle that for you!

Finally, two's complement

- First, we **flip all the bits**
  - Just like with 1's complement
- **Then add 1.**

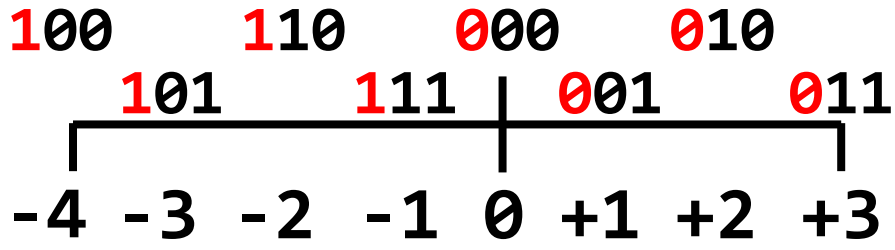
- The number line looks a little bit stranger
  - But there is only one 0, and it *is* 0!



Finally, two's complement

- First, we **flip all the bits**
  - Just like with 1's complement
- **Then add 1.**

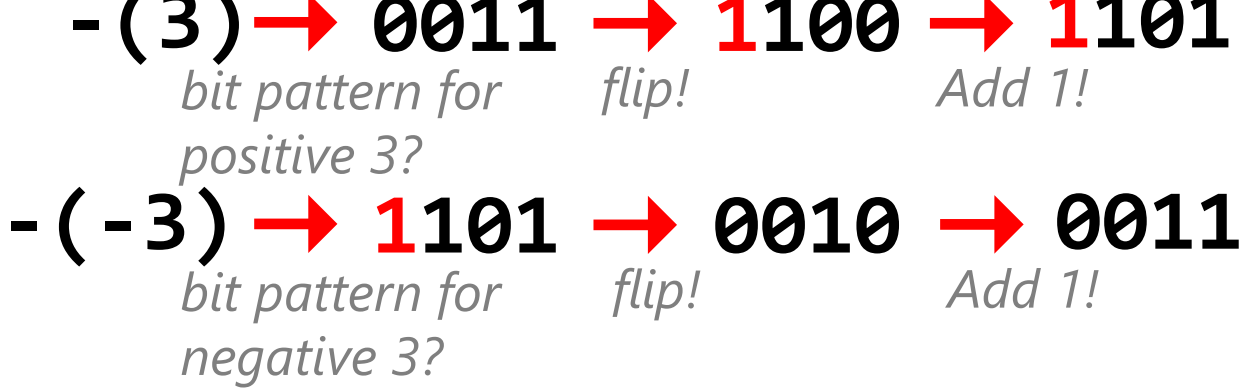
- The number line looks a little bit stranger
  - But there is only one 0, and it *is* 0!



- It's lopsided: **There is no +4!**
  - But arithmetic is easy!
  - when someone says "signed," 99% of the time they mean this
  - When **I** say "signed", I 100% of the time mean this

Sign-magnitude arithmetic

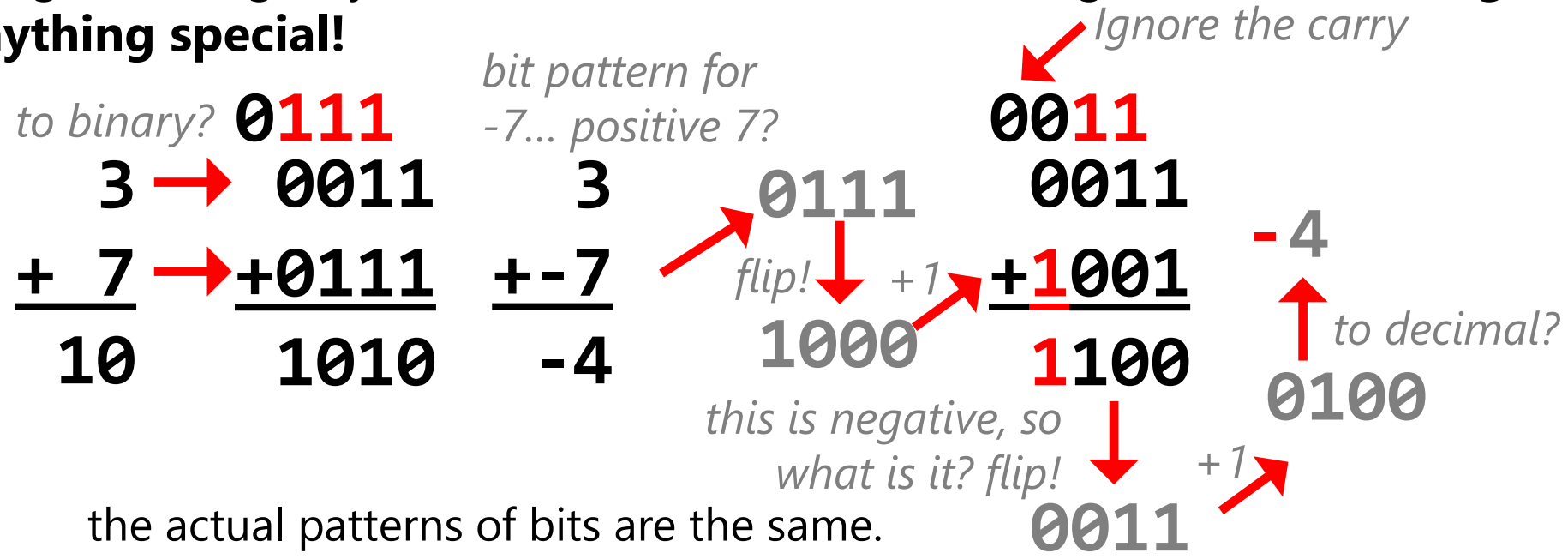
- Negation



- You don't need to subtract!!
  - flip(k)+1 == flip(k-1)
    - If you ignore the carry! ☺

Two's complement addition

- the great thing is: you can add numbers of either sign **without having to do anything special!**

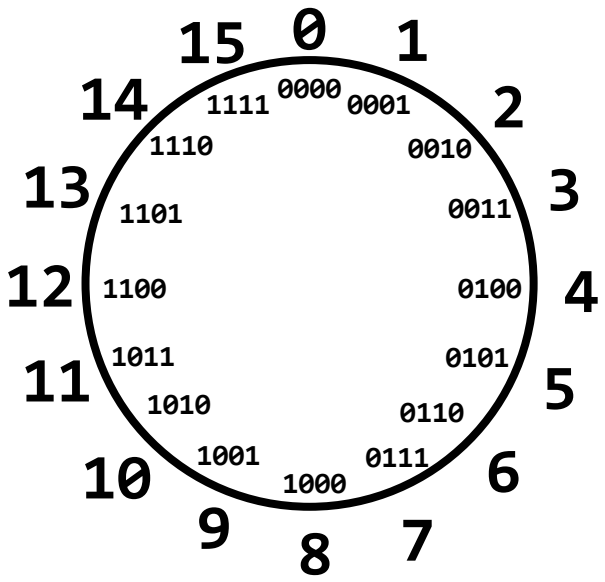


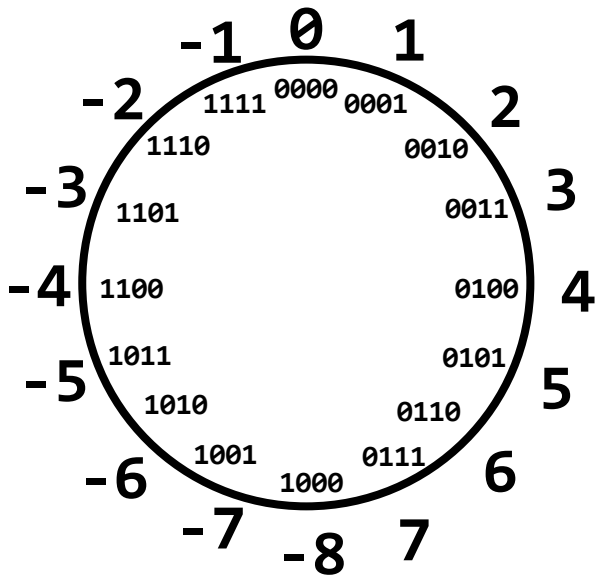
the actual patterns of bits are the same.
so how does the computer "know" whether it's doing signed or unsigned addition?

IT DOESN'T

This is how it looks

- Interpreting bit patterns as (left) unsigned and (right) signed





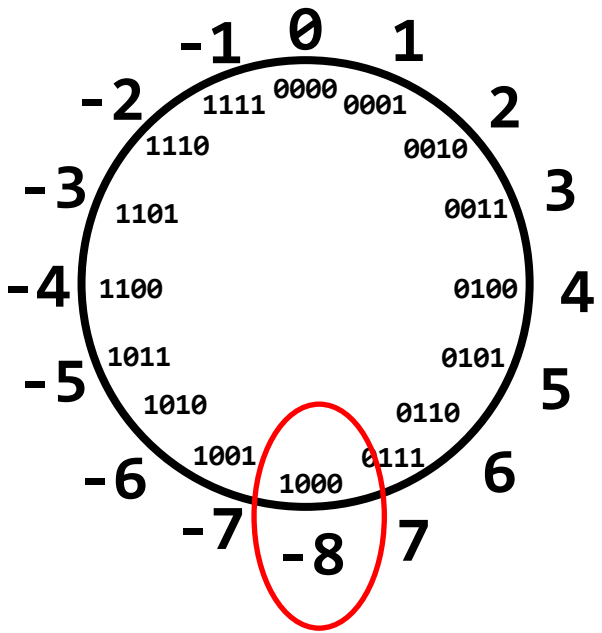
Two's complement negation

- (assuming 4 bits) What about??? -8?
  - What happens if we negate -8?
    - i.e. -(-8)?

-(-8) → **1000** → **0111** → **1000** → **-8???**

bit pattern for negative 8? *flip!* *Add 1!*

WHAT DID YOU DO WRONG??????



Absolutely Bonkers – Slide borrowed from wilkie (449)

```
public class AbsTest {
  public static int abs(int x) {
    if (x < 0) {
      x = -x;
    }
    return x;
  }

  public static void main(String[] args) {
    System.out.println(
      String.format("|%d| = %d", Integer.MIN_VALUE, AbsTest.abs(Integer.MIN_VALUE))
    );
  }
}
```



// Outputs: |-2147483648| = -2147483648

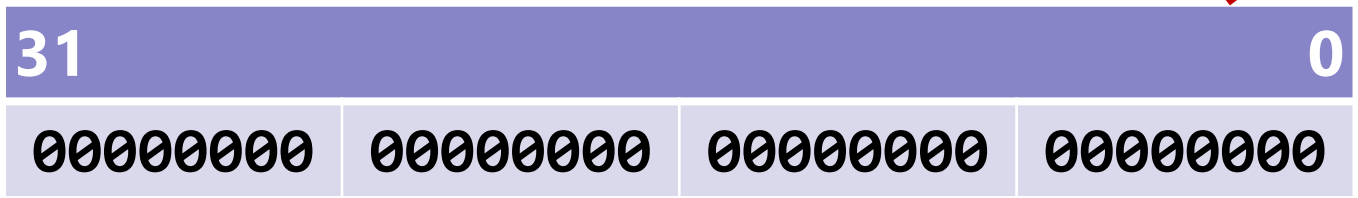
Q: How many bits is a Java int? What happened here?

Neat properties

- If we take a **positive** number and **add zeros**:
  - $120_{10} = 0111\ 1000_2$
  - $0000\ 0000\ 0111\ 1000_2 = 120_{10}$
- If we take a **negative** number and **add ones**:
  - $1000\ 0111_2 = - 0111\ 1000_2 + 1_2$
    $= - 0111\ 1001_2 = -121_{10}$
  - $1111\ 1111\ 1000\ 0111_2 = - 0000\ 0000\ 0111\ 1000_2 + 1_2$
    $= - 0000\ 0000\ 0111\ 1001_2$
    $= - 121_{10}$

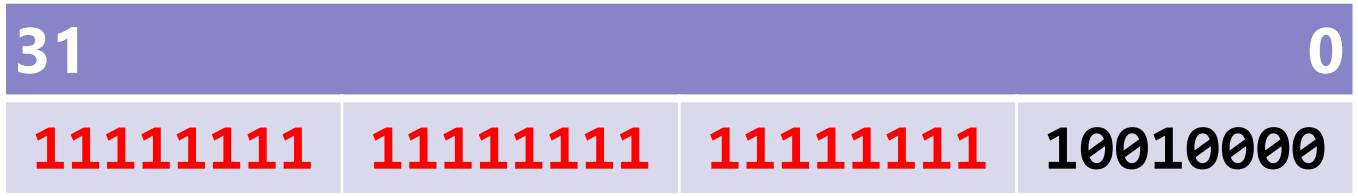
A return to EXPAND VALUE

- if you load a **byte…**



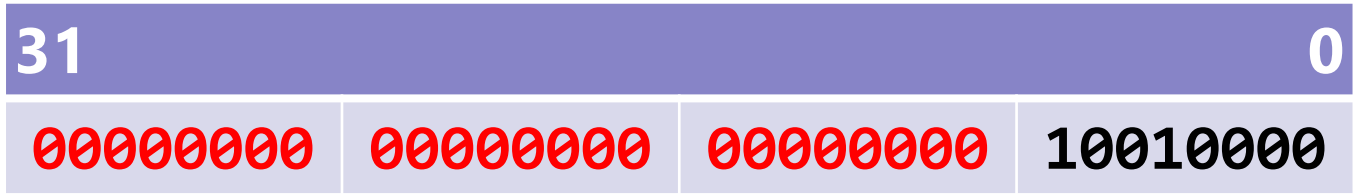
10010000

If the byte is **signed…** what *should* it become?



`lb` does sign extension.

If the byte is **unsigned…** what *should* it become?



`lbu` does zero extension.

Overflow

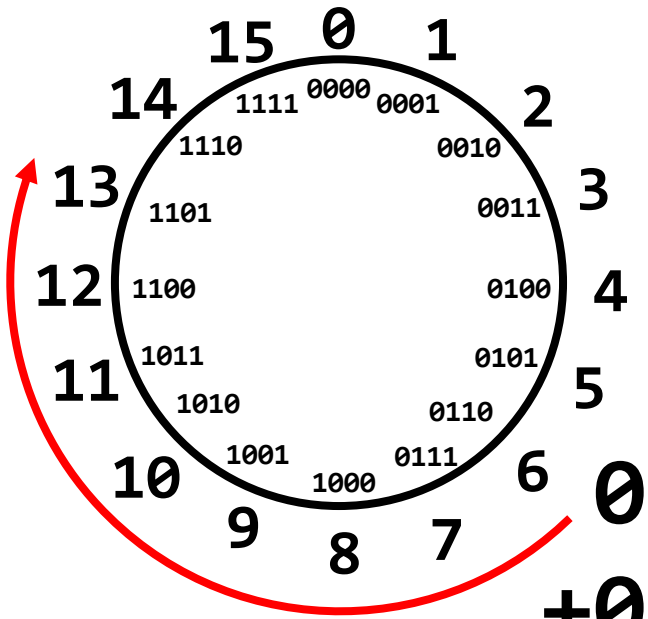
Overflow



- In computers, **numbers are finite.**
- Let's say our 4-digit display was counting up: 9997, 9998, 9999…
- What comes "next"?
  - What does this "0000" *really mean?*
  - It **wrapped around.**
- This is **overflow:** the number you are *trying* to represent is **too big to be represented.**
- Essentially, **all arithmetic on the computer is modular arithmetic!**
  - This causes a *lot* of software bugs.
  - https://en.wikipedia.org/wiki/Pac-Man#Level_256

Number carrousel

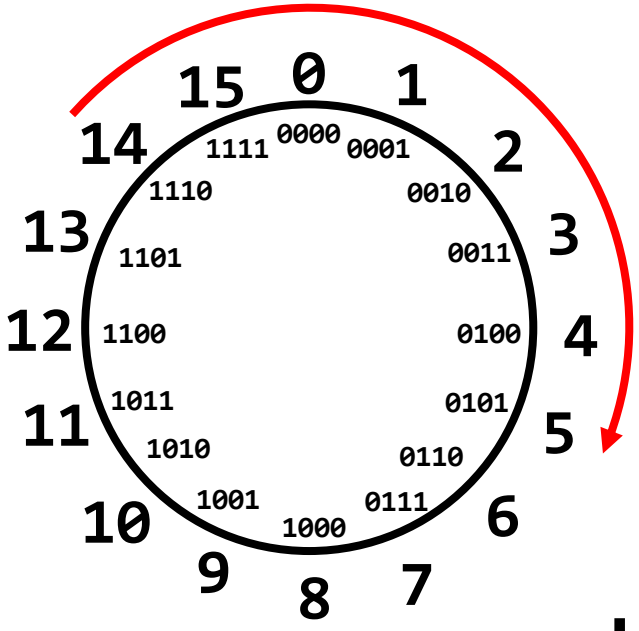
- Computers perform modulus arithmetic
  - Meaning: it goes around!
  - E.g. in a 4-bit computer




```
  0110
+0111
------
  1101
```

what is 6 + 7?

Unsigned overflowing



Uhhhhh

what is 14 + 7?

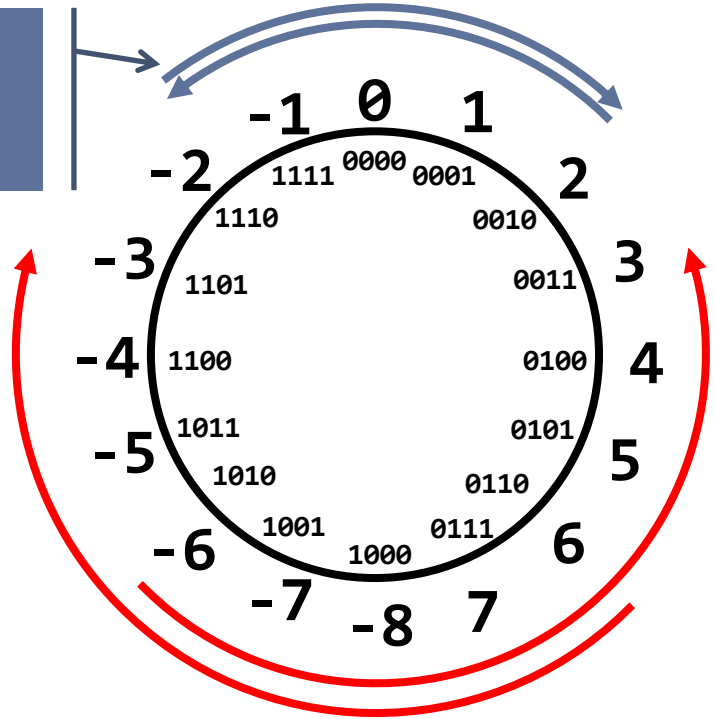
```
 1110
+0111
------
10101
```

If the result is smaller than either addend, there is an overflow

Signed overflowing

- It may also happen with signed numbers

This is OK!



The sign of overflow is the opposite sign

If the result of adding two numbers with the **same sign** results in the **opposite sign**, there is overflow

what is -6 - 7?

```
  1 0 0 0
  1010
+1001
------
  0011
```

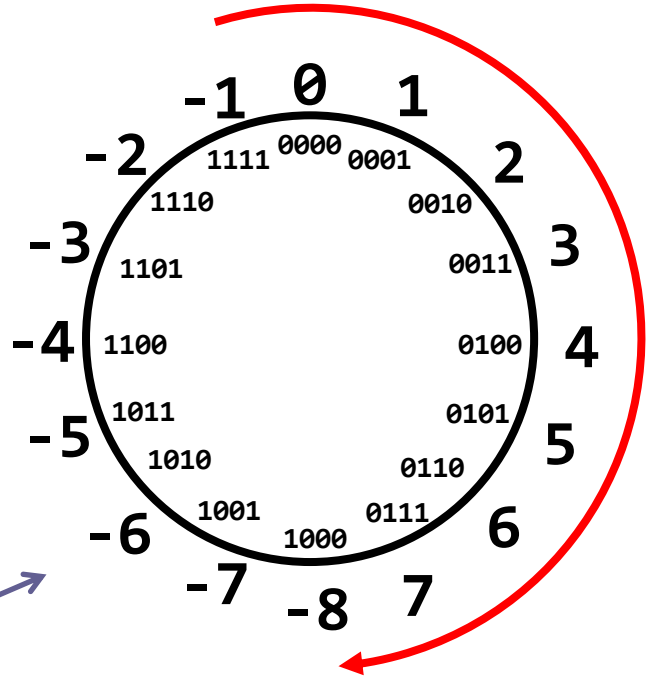
what is 6 + 7?

```
  0 1 1 0
  0110
+0111
------
  1101
```

Signed overflowing

What about this?

How can we detect if operations with different signs overflow?



This is impossible:
Max positive = 7
-1+7=6!

How many bits?

- if you add two 2-digit decimal numbers, what's the **largest number** you can get?
- what about two 4-digit decimal numbers?
- what about two 4-*bit* numbers?
- what's the pattern of the **number of digits?**
  - if you **add** two *n*-digit numbers *in any base*…
  - the result will have **at most *n* + 1 digits**
- that means if we add two **32-bit** numbers…
  - …we might get a **33-bit** result!
  - if we have more bits than we can store in our number, that's **overflow.**

```
 99   9999
+99  +9999
---  -----
198  19998
```

```
 1111
+1111
-----
11110
```

Q: How many bits do you need to represent -16 in 1s' complement?

Q: How many bits do you need to represent -16 in 2's complement?

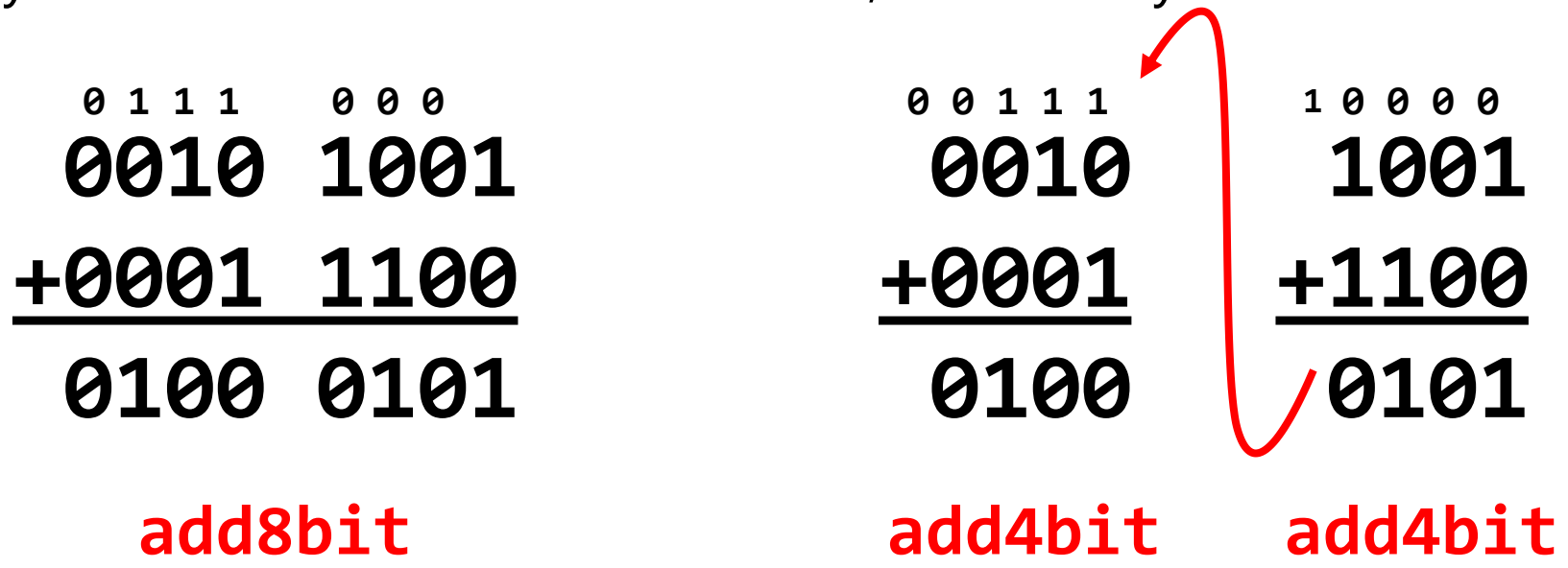
Handling overflow

- we could ***ignore it***
  - in MIPS: **addu, subu**
  - this is usually a **bad idea**
    - your program is broken
  - it's also the default in most languages, thanks C
- we could fall on the **floor - *i.e. crash***
  - in MIPS **add, sub**
  - can be handled and recovered from
  - but more complex
- we could ***store*** *that 33rd bit* ***somewhere else***



Maybe the bit bucket is a real place…

- many *other* architectures do this
  - MIPS does not.
- they have a "carry bit" register
  - this can be checked by the program after an add/sub
- this is very useful for **arbitrary precision arithmetic**
  - if you want to add 2048-bit numbers, chain many 32-bit additions



More Bonkers

```
public class Main {
  public static void main(String []args) {
     int w = Integer.MAX_VALUE;

     if (w + 1 > w) {
        System.out.println("This does not happen.\n");
     }

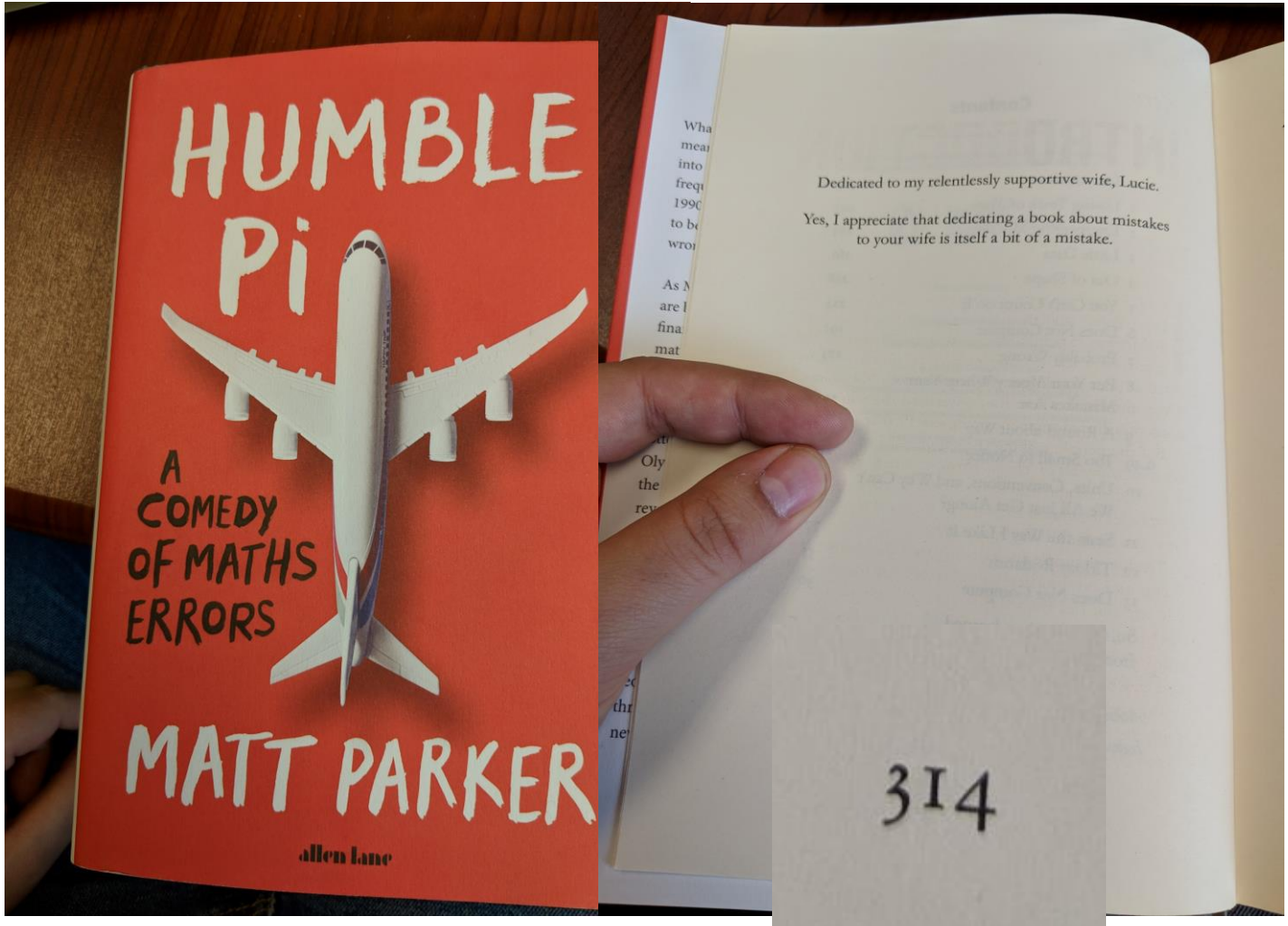
     int z = w + 1;
     if (z > w) {
        System.out.println("This does not happen.\n");
     }
     System.out.println("The end.\n");
  }

}
```



I have a book

What's the first page?

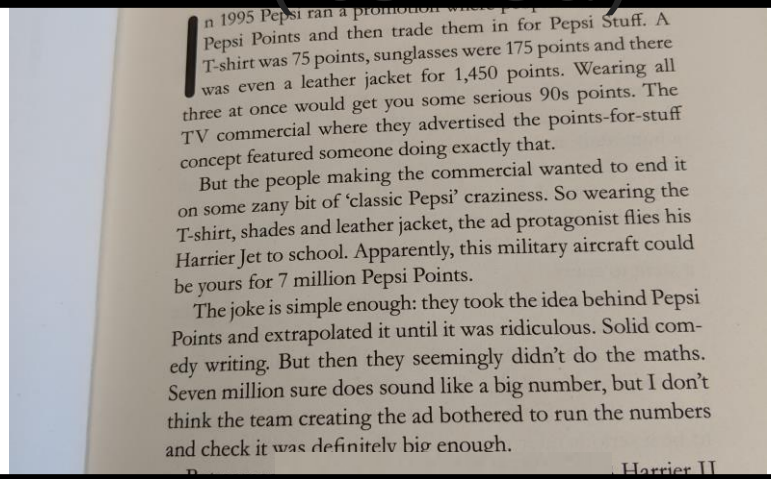


Hilarious, right?

Then…

What follows, chapter 1?

(it's not!)



In 1995 Pepsi ran a promotion where [illegible] Pepsi Points and then trade them in for Pepsi Stuff. A T-shirt was 75 points, sunglasses were 175 points and there was even a leather jacket for 1,450 points. Wearing all three at once would get you some serious 90s points. The TV commercial where they advertised the points-for-stuff concept featured someone doing exactly that.

But the people making the commercial wanted to end it on some zany bit of 'classic Pepsi' craziness. So wearing the T-shirt, shades and leather jacket, the ad protagonist flies his Harrier Jet to school. Apparently, this military aircraft could be yours for 7 million Pepsi Points.

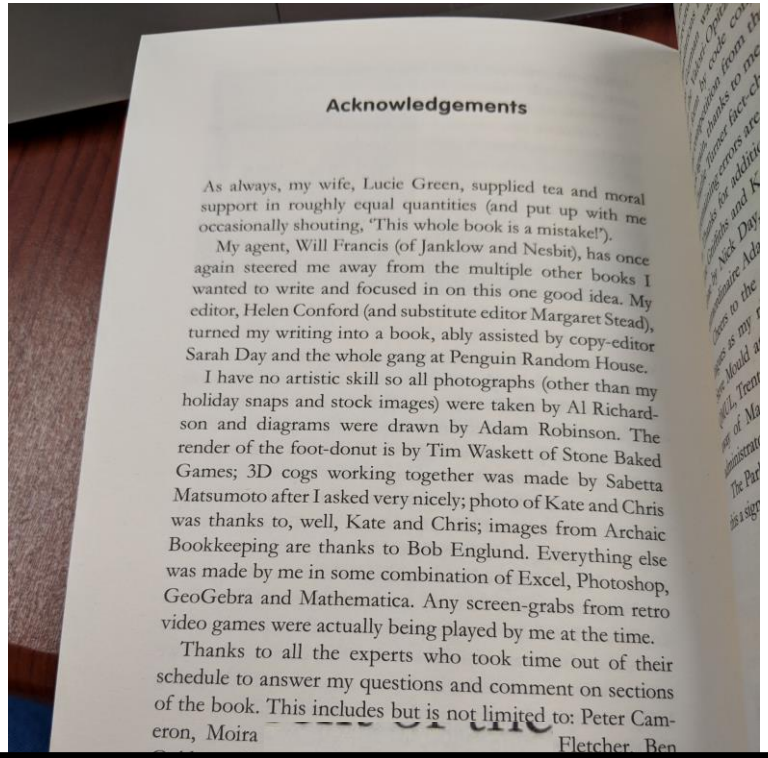
The joke is simple enough: they took the idea behind Pepsi Points and extrapolated it until it was ridiculous. Solid comedy writing. But then they seemingly didn't do the maths. Seven million sure does sound like a big number, but I don't think the team creating the ad bothered to run the numbers and check it was definitely big enough.

Harrier Jet

What number page number does it start at?

(it's not "2"!)

Is it zero?



Acknowledgements

As always, my wife, Lucie Green, supplied tea and moral support in roughly equal quantities (and put up with me occasionally shouting, 'This whole book is a mistake!').

My agent, Will Francis (of Janklow and Nesbit), has once again steered me away from the multiple other books I wanted to write and focused in on this one good idea. My editor, Helen Conford (and substitute editor Margaret Stead), turned my writing into a book, ably assisted by copy-editor Sarah Day and the whole gang at Penguin Random House.

I have no artistic skill so all photographs (other than my holiday snaps and stock images) were taken by Al Richardson and diagrams were drawn by Adam Robinson. The render of the foot-donut is by Tim Waskett of Stone Baked Games; 3D cogs working together was made by Sabetta Matsumoto after I asked very nicely; photo of Kate and Chris was thanks to, well, Kate and Chris; images from Archaic Bookkeeping are thanks to Bob Englund. Everything else was made by me in some combination of Excel, Photoshop, GeoGebra and Mathematica. Any screen-grabs from retro video games were actually being played by me at the time.

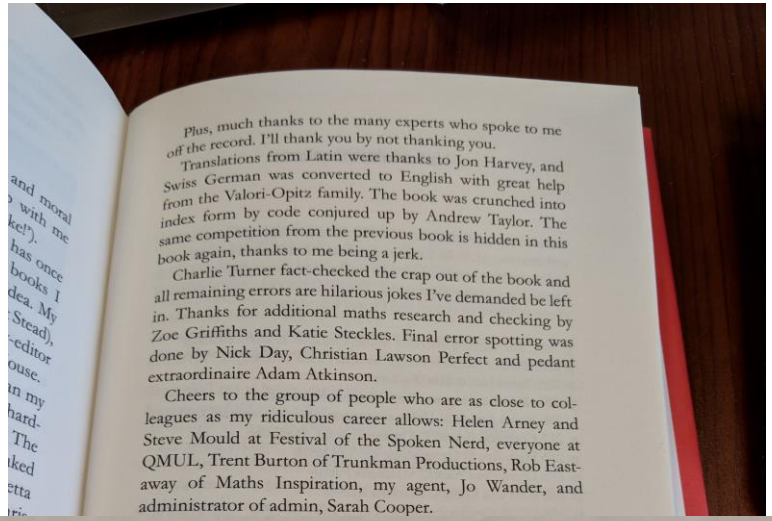
Thanks to all the experts who took time out of their schedule to answer my questions and comment on sections of the book. This includes but is not limited to: Peter Cameron, Moira

Fletcher, Ben

Where does it end?

(nope!)

Yikes....



Plus, much thanks to the many experts who spoke to me off the record. I'll thank you by not thanking you.

Translations from Latin were thanks to Jon Harvey, and Swiss German was converted to English with great help from the Valori-Opitz family. The book was crunched into index form by code conjured up by Andrew Taylor. The same competition from the previous book is hidden in this book again, thanks to me being a jerk.

Charlie Turner fact-checked the crap out of the book and all remaining errors are hilarious jokes I've demanded be left in. Thanks for additional maths research and checking by Zoe Griffiths and Katie Steckles. Final error spotting was done by Nick Day, Christian Lawson Perfect and pedant extraordinaire Adam Atkinson.

Cheers to the group of people who are as close to colleagues as my ridiculous career allows: Helen Arney and Steve Mould at Festival of the Spoken Nerd, everyone at QMUL, Trent Burton of Trunkman Productions, Rob Eastaway of Maths Inspiration, my agent, Jo Wander, and administrator of admin, Sarah Cooper.

What can possible come after zero?

(How much is 0-1?)